



北京邮电大学
Beijing University of Posts and Telecommunications



Queen Mary
University of London

Undergraduate Project Report 2025/26

Network Resource Management Based on Language Model Multi-Agent Systems

Name: Zheyun Zhao
School: International School
Class: 2022215124
QMUL Student No.: 221170559
BUPT Student No.: 2022213670
Programme: Intelligent Science and Technology

Date: 13-04-2026

Table of Contents

Abstract	3
Keywords	3
Chapter 1: Introduction	5
1.1 Background and Motivation	5
1.2 Problem Statement	5
1.3 Research Objectives	6
1.4 Scope and Assumptions	6
1.5 Contributions	7
1.6 Report Organisation	7
Chapter 2: Background	8
2.1 Network Slicing Architecture and Resource Management Models	8
2.2 SLA-Aware Scheduling and Two-Tier Control Mechanisms	9
2.3 LLM-Based Agent Frameworks in Network Management	9
2.4 Multi-Agent Coordination and Negotiation Mechanisms	10
2.5 Identified Research Gaps	11
Chapter 3: Design and Implementation	12
3.1 System Overview and Architectural Principles	12
3.2 Two-Tier Scheduling Framework	13
3.3 Agent Role Definition and Internal Logic	14
3.4 Proposal--Evaluation--Negotiation Protocol	15
3.5 Gymnasium-Based Simulation Environment	16
3.6 LangGraph-Orchestrated Agent Workflow	17
3.7 Baseline Methods and Implementation Details	17
Chapter 4: Results and Discussion	20
4.1 Experimental Design and Evaluation Metrics	20
4.2 Core Performance Comparison	20
4.3 Negotiation Mechanism Validation	20
4.4 Result Analysis	20
4.5 Threats to Validity and Limitations	20
4.6 Practical Implications	20
Chapter 5: Conclusion and Further Work	21
5.1 Conclusion	21
5.2 Reflection	21

5.3 Further Work	21
<i>References</i>	22
<i>Acknowledgment</i>	24
<i>Appendices</i>	25
Disclaimer.....	25
Acknowledgement of using Generative AI.....	26
Project specification	27
Early-term progress report.....	28
Mid-term progress report	29
Supervision log	30
Prompt Templates for Each Agent Role.....	31
Agent Message Format (JSON Schema)	34
Simulation Environment Parameters	37
DRL Training Convergence Curves	38
<i>Risk and Environmental Impact Assessment</i>	39

Abstract

The abstract should be a short overview of the whole report (200-300 words maximum). It should give the reader enough information about your entire project to know what you have tried to do and whether you were successful.

Keywords

Keyword 1, Keyword 2, Keyword 3 (List of Keywords, separated by commas)

摘要

This is the Chinese translation of the Abstract.

关键词

This is the Chinese translation of the Keywords, separated by commas and following the same sequence.

Chapter 1: Introduction

1.1 Background and Motivation

The fifth-generation (5G) mobile network has shifted from the homogeneous “one network fits all” paradigm of previous generations towards a service-oriented architecture in which a shared physical infrastructure is partitioned into multiple logical networks through the use of network slicing (1). Each slice is designed to satisfy the differentiated Quality of Service (QoS) requirements of a specific service class. The three canonical slice types defined by 3GPP – Enhanced Mobile Broadband (eMBB), Ultra-Reliable Low-Latency Communication (URLLC), and massive Machine-Type Communication (mMTC) – exhibit fundamentally different performance requirements: eMBB prioritises sustained high throughput, URLLC targets millisecond-level end-to-end latency together with stringent reliability guarantees, and mMTC is designed to accommodate the simultaneous access of a very large number of low-rate devices.

When these slice types coexist on a common pool of radio resources, their objectives become inherently competitive. The bandwidth consumed by a population of eMBB users streaming high-definition video may directly erode the latency budget available to URLLC flows, while bursty mass reporting by mMTC devices can exhaust access resources that other slices rely upon (1). Traditional approaches to resource management – rule-based heuristics and Deep Reinforcement Learning (DRL)-based optimisation – have been widely studied in this setting (2, 3). Although these methods can encode hard constraints and priorities effectively, they struggle to integrate Service Level Agreement (SLA) semantics, operator policies and dynamically evolving user behaviour into a single adaptive decision-making framework.

The emergence of Large Language Models (LLMs) and the broader family of LLM-based autonomous agents has introduced an alternative line of investigation (4, 5). Recent work has begun to examine the use of LLMs as high-level decision engines for wireless network management, motivated by their capacity for in-context reasoning, structured tool use and natural-language policy expression (6, 7). However, the application of LLM-based agents to network slicing remains at an early stage, and a number of architectural questions have not yet been addressed in a satisfactory manner.

1.2 Problem Statement

Two gaps in the current literature motivate the present study.

Gap 1: Absence of an end-to-end multi-agent LLM framework for network slice resource management. The WirelessAgent framework (6) has demonstrated that an LLM endowed with a perception–memory–planning–action cognitive architecture can approach rule-optimal performance in slicing scenarios without any task-specific training. Nevertheless, WirelessAgent

remains, at its core, a centralised single-agent design: one LLM is responsible for resource decisions across all slices, with no explicit specialisation of analytical roles or structured mechanism for mediating between the conflicting objectives of different slices. CommLLM (8) introduces a three-component multi-agent architecture, but its validation is primarily targeted at semantic communication tasks rather than slice resource allocation. A complete framework dedicated to network slice resource management, supporting explicit multi-agent collaboration and validated end-to-end in a reproducible slicing environment, therefore remains underexplored.

Gap 2: Lack of structured negotiation and conflict-resolution mechanisms. Network slice resource allocation can be cast as a constrained multi-party optimisation problem in which total radio resources are fixed and the slices behave as competing stakeholders (9). This setting differs substantially from the predominantly cooperative task structures addressed by general-purpose multi-agent LLM frameworks such as MetaGPT (10) or ChatDev (11). In competitive slicing scenarios, agents must reconcile conflicting objectives under hard resource constraints and service-level requirements, yet systematic negotiation protocols tailored to such environments are rarely discussed in the existing network management literature.

1.3 Research Objectives

The overarching objective of this project is to design, implement and evaluate an LLM multi-agent negotiation-based resource allocation framework for 5G network slicing, in which a low-frequency LLM multi-agent policy layer is coupled with a high-frequency rule-based execution layer, and to compare this framework with rule-based and DRL baselines in a reproducible simulation environment. This high-level objective is refined into two research questions.

RQ1: Can a multi-agent LLM framework outperform traditional rule-based strategies and DRL methods in network slice resource management? The associated objective is to quantify the performance differences between the proposed framework and representative baselines in terms of SLA satisfaction rate, resource utilisation and dynamic load adaptability.

RQ2: Does the negotiation mechanism between agents yield a structural advantage, as opposed to simply providing additional LLM computation? The associated objective is to compare a configuration that includes the full proposal–evaluation–negotiation protocol against an otherwise identical configuration in which the coordinator imposes allocations directly, in order to isolate the contribution of negotiation itself.

1.4 Scope and Assumptions

The scope of this work is deliberately constrained in order to ensure that the research questions can be addressed rigorously within the time available. The study is confined to a simulation-based evaluation using a custom Gymnasium (12) environment; no deployment on physical ra-

dio hardware or operational testbed is attempted. Three slice types – eMBB, URLLC and mMTC – are considered, following the canonical 3GPP classification. The LLM backbone is provided by a single commercial API (GPT-4) and the temperature is fixed to zero for all experiments in order to constrain reproducibility to environmental and DRL stochasticity. SLA definitions adopted in the simulation represent relative priority isolation rather than strict physical-layer 3GPP metrics; the implications of this abstraction are discussed explicitly in Chapter 3. Finally, the workload patterns used in the experiments are synthetic, reflecting the scarcity of publicly available slicing workload datasets with explicit SLA annotations.

1.5 Contributions

The contributions of this project are twofold.

1. **A reproducible 5G network slicing simulation environment.** A custom Gymnasium-based environment is implemented, with clearly defined state and action spaces, a parameterised reward function and per-slice SLA evaluation logic. All parameters and interfaces are documented to support reproduction and extension.
2. **A two-tier LLM multi-agent decision framework with an explicit negotiation protocol.** The upper tier consists of specialised slice-manager agents and a global coordinator agent that collaborate on resource allocation decisions through a three-phase proposal–evaluation–negotiation protocol; the lower tier executes rule-based cached strategies at millisecond granularity. The framework is compared against rule-based, DRL and single-agent LLM baselines through a set of controlled experiments designed to address RQ1 and RQ2.

1.6 Report Organisation

The remainder of this report is organised as follows. Chapter 2 reviews the technical background and related work on network slicing architectures, SLA-aware scheduling, LLM-based agent frameworks for network management, and multi-agent coordination and negotiation mechanisms, and identifies the specific research gaps addressed in this work. Chapter 3 presents the design and implementation of the proposed framework, including the two-tier scheduling architecture, the internal logic of each agent role, the negotiation protocol, the Gymnasium-based simulation environment, the LangGraph-orchestrated workflow, and the baseline methods used for comparison. Chapter 4 describes the experimental design and reports the results obtained on the core performance comparison and the negotiation mechanism validation, together with a discussion of threats to validity and practical implications. Chapter 5 concludes the report with a reflection on the outcomes and an outline of possible directions for further work.

Chapter 2: Background

This chapter provides the technical context in which the proposed framework is situated. It reviews the principles of network slicing and the associated resource management models, discusses SLA-aware scheduling and two-tier control, surveys the emerging body of work on LLM-based agents for network management, examines general and network-specific multi-agent coordination mechanisms, and concludes by identifying the research gaps that motivate the design presented in Chapter 3.

2.1 Network Slicing Architecture and Resource Management Models

Network slicing is one of the defining capabilities of 5G and beyond-5G networks. As surveyed by Afolabi *et al.* (1), slicing partitions a shared physical infrastructure into multiple end-to-end logical networks, each tuned to the requirements of a specific service class. The 3GPP service classification distinguishes eMBB, URLLC and mMTC, which differ substantially in their throughput, latency, reliability and connection-density targets. Because these slice types share a common pool of radio resources, resource allocation across slices must simultaneously respect hard capacity constraints and the heterogeneous performance objectives of each tenant.

Two families of methods have been applied extensively to this problem. Rule-based and heuristic allocation schemes encode priorities and constraints explicitly and are attractive for their interpretability and bounded computational cost, but they generalise poorly to traffic patterns that diverge from those anticipated at design time. Learning-based approaches, and in particular DRL-based resource managers, have attracted considerable attention. Li *et al.* (2) formulated bandwidth allocation across slices as a Markov decision process and demonstrated that DRL agents can outperform hand-crafted heuristics under non-stationary demand. Sciancalepore *et al.* (3) proposed the RL-NSB network slice broker, which uses reinforcement learning to admit and dimension slice requests so as to balance revenue and SLA compliance. Caballero *et al.* (9) analysed slicing under a game-theoretic lens, showing that slice customisation can be supported even when tenants behave strategically. More recent work, such as Nasser and Alani (13), has explored hybrid deep-learning architectures for slicing, reinforcing the observation that learning-based methods can adapt to complex workload mixtures but require large amounts of labelled data and careful reward engineering.

A common limitation of these approaches is that the decision-making component is monolithic: a single policy is trained or hand-crafted to output allocations for all slices at once, and neither the explicit articulation of the objectives of each slice nor a structured way of reconciling conflicts between them is provided by the architecture itself.

2.2 SLA-Aware Scheduling and Two-Tier Control Mechanisms

Slice resource management must operate simultaneously at two very different timescales. At the radio interface, scheduling decisions are taken on a per-Transmission Time Interval (TTI) basis – typically every one or two milliseconds – and must take into account instantaneous channel quality, buffer occupancy and user fairness. At a longer timescale, inter-slice resource quotas, admission control decisions and policy updates are driven by aggregated performance indicators such as SLA satisfaction rates, tenant-level throughput and long-term utilisation.

This separation has led to the adoption of two-tier or hierarchical control architectures in which fast rule-based or model-based controllers handle per-TTI decisions, while slower optimisation or learning-based components adjust the parameters of the lower tier. Such architectures are compatible with the hierarchy already present in modern radio access networks, in which Medium Access Control (MAC)-layer schedulers operate at millisecond granularity while Radio Resource Management (RRM) functions revise resource partitions on a coarser timescale. In the context of LLM-based decision making, the two-tier view is particularly relevant because LLM inference latency is typically on the order of hundreds of milliseconds to seconds, which makes it infeasible to place an LLM directly on the per-TTI control loop. Recent work on LLM-augmented O-RAN control (14) and on dual-loop LLM-based management (15) has therefore advocated separating a slow semantic reasoning loop from a fast execution loop, foreshadowing the two-tier design adopted in this project.

2.3 LLM-Based Agent Frameworks in Network Management

Between 2023 and 2025, research on LLMs for wireless networks has developed along three broadly distinct technical routes, each making different trade-offs in how domain knowledge is injected into the model.

The first route relies on prompt engineering and explicit agent workflows. WirelessAgent (6) decomposes network management into a sequential workflow of intent understanding, slice allocation, bandwidth allocation and load balancing, where each stage is implemented as a LangGraph node that manages global state transitions. Its main contribution is to demonstrate that a cognitive architecture consisting of perception, memory, planning and action, combined with tool calling, can approach rule-optimal performance in slicing scenarios without any task-specific training – the reported bandwidth utilisation is only a few percentage points below the rule-optimal baseline under the considered experimental configuration. WirelessLLM (7) proposes a complementary methodological roadmap, combining prompt-based knowledge alignment, Retrieval-Augmented Generation (RAG) (16), fusion with specialised models and continual learning, as a general recipe for adapting LLMs to wireless intelligence tasks.

The second route addresses domain adaptation through parameter fine-tuning. NetLLM (17)

reformulates a selection of networking problems – including viewport prediction, adaptive bitrate selection and cluster job scheduling – as sequential decision problems and fine-tunes a Llama2-based backbone using low-rank adaptation, reporting improvements over DRL baselines on these tasks. While effective, this route requires substantial labelled data, a condition that is not easily satisfied in the slicing domain.

The third route explores cloud-edge collaboration. NetGPT (18) proposes an architecture in which lightweight LLMs are deployed at the edge for latency-sensitive inference while larger models remain in the cloud for global optimisation, anticipating the deployment challenges posed by the size and inference cost of frontier models. Related work by Chergui *et al.* (19) has investigated collective memory and knowledge sharing across LLM-based agents for cross-domain 6G management.

Across all three routes, the decision-making agent is typically centralised: a single LLM or LLM-powered component is responsible for the entire resource management task. This observation underlies one of the research gaps identified later in this chapter.

2.4 Multi-Agent Coordination and Negotiation Mechanisms

In parallel with the network-specific work reviewed above, general-purpose multi-agent LLM systems have become an active area of research. MetaGPT (10) encodes Standard Operating Procedures as prompt sequences and enforces the exchange of structured intermediate artefacts between agents, reducing the information loss that can arise when free-form natural language is the sole channel of communication. AutoGen (20) provides a flexible conversational programming paradigm in which agents with different roles coordinate through multi-turn dialogues, and has been applied to a wide range of tasks including code generation and data analysis. CAMEL (21) investigates the emergence of cooperative behaviour in role-playing agent societies, while ChatDev (11) applies multi-agent collaboration to software development by assigning dedicated roles to designers, coders and testers. A survey by Wang *et al.* (5) provides a broader taxonomy of LLM-based autonomous agents and their cognitive components.

These frameworks have matured rapidly, but they share an important assumption: the underlying task is predominantly cooperative and admits a natural division of labour. Adapting them to network resource management raises three specific difficulties. First, slice resource allocation is a competitive constrained optimisation problem in which the total bandwidth is fixed and agents representing different slices must compete for a limited pool of resources. Second, the decisions are subject to strong real-time constraints, which complicates the use of unbounded multi-turn dialogues. Third, decision outcomes are associated with clear quantitative evaluation criteria – SLA satisfaction rates, throughput, latency – which leave little room for the qualitative judgement that dominates many general-purpose multi-agent benchmarks.

Multi-agent attempts that target networking tasks directly remain limited. CommLLM (8) pro-

poses a three-component multi-agent architecture for 6G communications, but the reported validation is primarily focused on semantic communication and does not constitute a full slicing evaluation. The dual-loop edge-terminal collaboration of Qu *et al.* (15) is architecturally informative in its treatment of an LLM-driven semantic loop coupled with a fast execution loop, yet coordination within that framework remains predominantly centralised. ORAN-GUIDE (14) adopts a division of labour in which an LLM provides semantic understanding and a reinforcement learning component handles online optimisation, which offers useful insight into how the LLM decision latency can be decoupled from network real-time requirements.

Chain-of-Thought prompting (22) is commonly used within these frameworks to encourage the model to reason step by step, and has been shown to improve performance on tasks that involve arithmetic or logical decomposition. Retrieval augmentation (16) is frequently adopted as a lightweight mechanism for injecting domain knowledge without modifying model parameters. Both techniques are relevant for the design presented in Chapter 3.

2.5 Identified Research Gaps

Taken together, the work reviewed above suggests two complementary opportunities that this project seeks to address.

Gap 1. Existing LLM-based network management frameworks either adopt a centralised single-agent design and do not expose explicit negotiation between slices (6, 7), or they propose multi-agent architectures whose validation is confined to communication tasks other than slice resource allocation (8). A complete framework dedicated to slice resource management, which supports explicit multi-agent collaboration and is validated end-to-end in a reproducible slicing environment, has not yet been presented in the literature accessible to the author.

Gap 2. Although general-purpose multi-agent LLM frameworks such as MetaGPT (10) and ChatDev (11) provide rich coordination mechanisms, these mechanisms are designed with co-operative tasks in mind. Slice allocation is better modelled as a constrained multi-party optimisation problem in which the objectives of different slices compete under a fixed capacity budget (9). Structured negotiation and conflict-resolution protocols tailored to this competitive setting have received limited attention in the network management literature.

The design described in the following chapter responds to these two gaps by combining a two-tier scheduling architecture with a set of specialised slice-manager agents and a global coordinator that interact through an explicit proposal–evaluation–negotiation protocol.

Chapter 3: Design and Implementation

This chapter presents the design of the proposed LLM multi-agent negotiation framework for 5G network slice resource management and describes its implementation. Section 3.1 outlines the overall system architecture and the guiding design principles. Section 3.2 details the two-tier scheduling framework that reconciles LLM inference latency with the real-time demands of radio resource management. Section 3.3 specifies the roles and internal logic of the individual agents. Section 3.4 defines the proposal–evaluation–negotiation protocol. Section 3.5 describes the Gymnasium-based simulation environment. Section 3.6 discusses the LangGraph-orchestrated workflow. Section 3.7 presents the baseline methods implemented for comparison.

3.1 System Overview and Architectural Principles

The proposed framework is organised into three loosely coupled layers, as illustrated in Figure 1. The *Agent Layer* defines the specialised agents and their configurations. The *Collaboration Layer* implements inter-agent negotiation and conflict resolution. The *Environment Layer* provides a standardised simulation interface that decouples the decision logic from the underlying simulator. The full system is orchestrated through LangGraph, with a commercial LLM API (GPT-4) serving as the reasoning backbone.

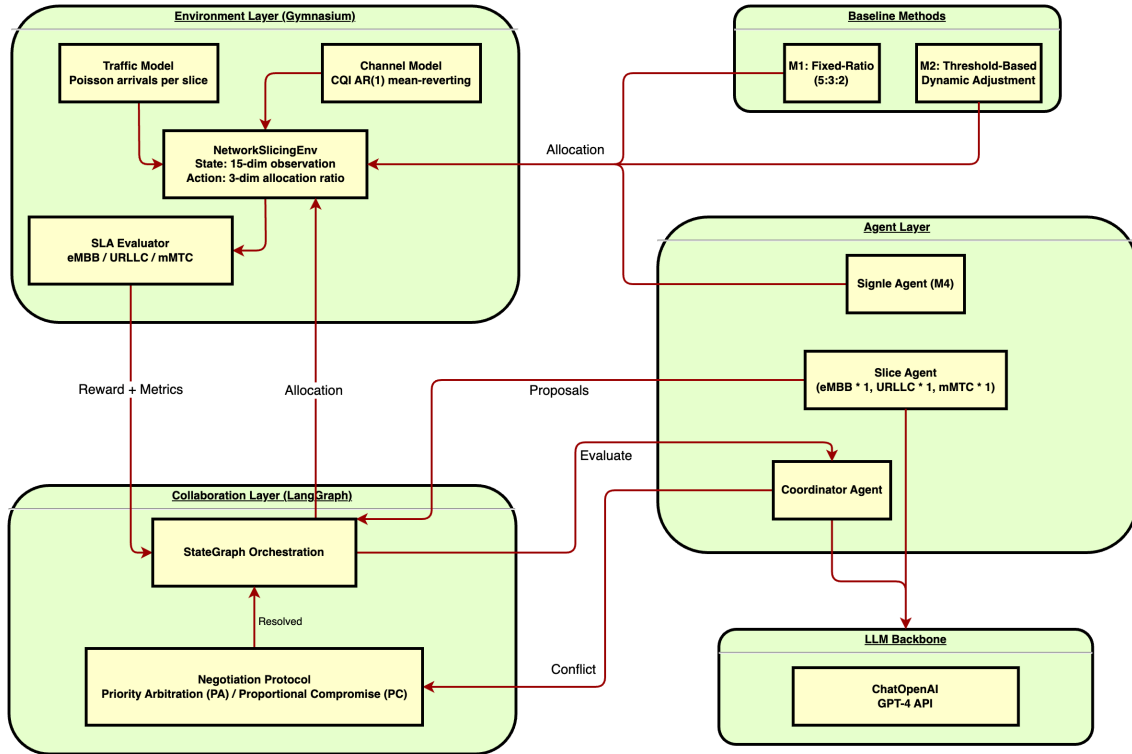


Figure 1: Overall system architecture of the proposed two-tier multi-agent framework.

The design is guided by four principles. *Separation of timescales*: slow semantic reasoning by

LLM agents is decoupled from fast rule-based execution so that LLM inference latency does not propagate into the per-TTI control loop. *Role specialisation*: each slice type is represented by a dedicated agent whose prompts and internal reasoning are tailored to the characteristics of that slice, while a global coordinator agent is responsible exclusively for cross-slice arbitration. *Structured communication*: inter-agent messages are exchanged as structured JSON artefacts rather than as free-form natural language, in the spirit of MetaGPT (10), in order to limit information loss and to make the behaviour of the system inspectable. *Graceful degradation*: whenever the LLM layer fails to produce a decision within a predefined latency budget, the system automatically falls back to a rule-based controller, so that the operational stability of the network is never contingent on the responsiveness of an external API.

3.2 Two-Tier Scheduling Framework

A central challenge addressed by the design is the order-of-magnitude gap between the inference latency of contemporary LLMs – typically hundreds of milliseconds to a few seconds per call – and the millisecond-level timescale at which radio scheduling decisions must be taken. Placing an LLM directly on the per-TTI control loop is therefore infeasible. Instead, the proposed framework adopts an explicit two-tier scheduling architecture, illustrated in Figure 2.

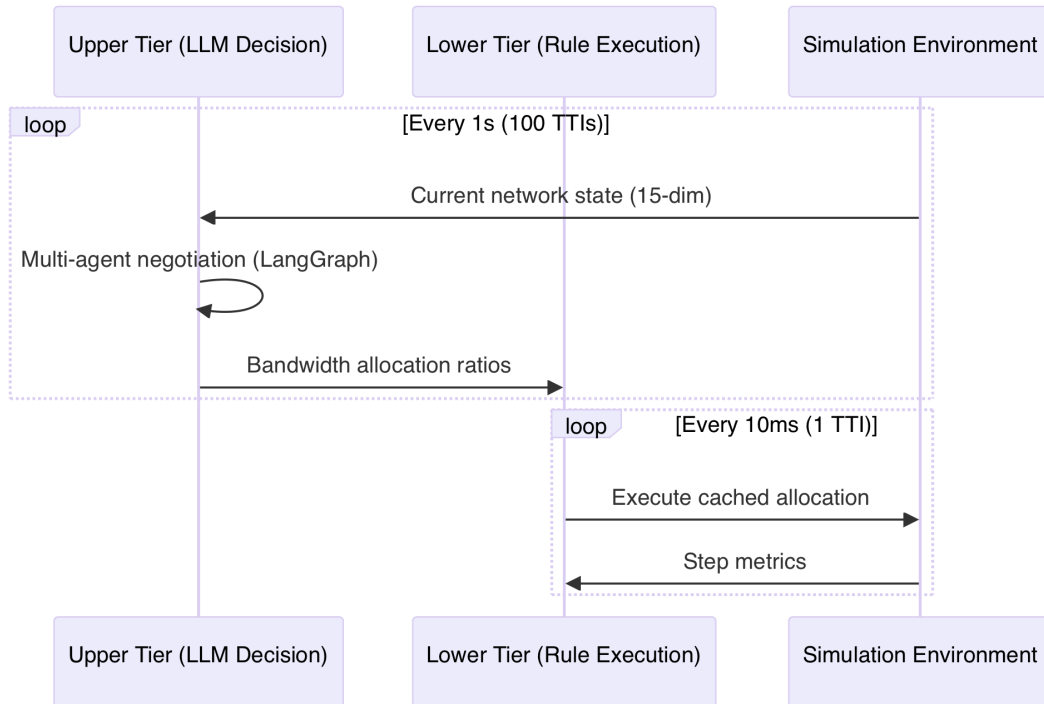


Figure 2: Two-tier scheduling: slow LLM policy updates at the upper tier and fast rule-based execution at the lower tier.

The *upper tier*, or LLM decision layer, is triggered once every 100 TTIs (i.e. approximately one second) in the baseline configuration. At each trigger, the slice-manager agents and the

global coordinator agent execute one round of the proposal–evaluation–negotiation protocol and update the inter-slice bandwidth allocation quotas. An emergency decision can additionally be triggered when the measured SLA violation rate exceeds a predefined threshold, so that the framework is able to react to sudden changes in traffic conditions.

The *lower tier*, or fast execution layer, operates at a TTI-level granularity of approximately 10 ms. Within each decision window it executes a cached rule-based strategy that enforces the inter-slice quotas produced by the upper tier, without invoking the LLM. This layer is responsible for per-user scheduling inside each slice and for monitoring SLA-related indicators that feed back into the upper tier.

A *maximum decision latency threshold* of 2 seconds is enforced on the upper tier. If the LLM layer fails to return a decision within this budget, the system automatically falls back to a rule-based allocation policy so that normal operation is preserved. This fallback behaviour is a direct consequence of the graceful degradation principle introduced in Section 3.1.

3.3 Agent Role Definition and Internal Logic

The proposed framework defines four agents: one slice-manager agent per slice type (eMBB, URLLC, mMTC) and a single global coordinator agent. Each agent shares a common internal structure consisting of three modules – *Perception*, *Planning* and *Action* – in line with the cognitive architecture pattern adopted by WirelessAgent (6) and surveyed more broadly by Wang *et al.* (5).

Slice Manager Agent. A slice-manager agent is instantiated for each of the three slice types. The Perception module converts the numerical state of the slice (user count, average CQI, buffer occupancy and current SLA satisfaction rate) into a structured natural-language description that is fed into the LLM prompt. The Planning module uses Chain-of-Thought prompting (22) to assess the current resource demand of the slice in light of its SLA targets. The Action module outputs a resource demand proposal consisting of a requested bandwidth share, a minimum acceptable share, a priority declaration and a short justification, all encoded in a structured JSON format.

Global Coordinator Agent. A single global coordinator agent is responsible for cross-slice arbitration. Its Perception module aggregates the proposals produced by the slice-manager agents together with the global resource state. The Planning module determines whether the proposals are jointly compatible – that is, whether the sum of the requested shares can be satisfied within the available bandwidth budget. If the proposals are compatible, they are approved directly. If a conflict is detected, the negotiation protocol described in Section 3.4 is triggered. The Action module executes the final bandwidth allocation scheme and invokes the correspond-

ing simulation environment API.

3.4 Proposal–Evaluation–Negotiation Protocol

When the coordinator detects that the aggregate demand exceeds the available capacity, a three-phase negotiation protocol is triggered. The overall message flow is shown in Figure 3.

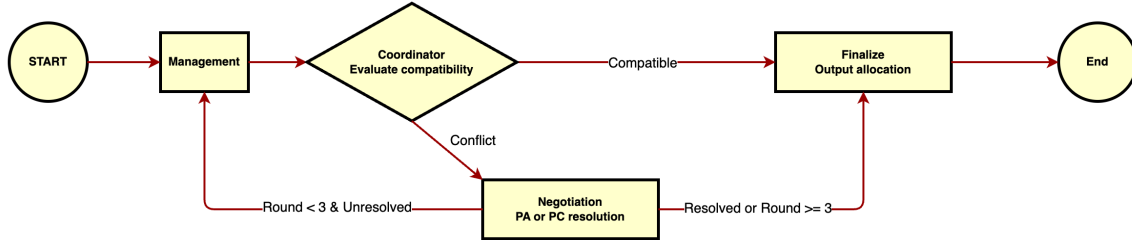


Figure 3: Message flow of the three-phase proposal–evaluation–negotiation protocol.

Phase 1 – Proposal. Each slice-manager agent independently evaluates its current demand and submits a resource proposal as a structured JSON message. The proposal contains the requested bandwidth share, the minimum acceptable share, a priority tag and a short natural-language justification that supports subsequent reasoning steps.

Phase 2 – Evaluation. The global coordinator agent evaluates the joint feasibility of the proposals. Feasibility checks include whether the sum of the requested shares exceeds the available bandwidth and whether any hard priority rules are violated (for instance, the convention that URLLC traffic is privileged over eMBB when its latency requirements are at risk). If the proposals are jointly feasible, they are approved and forwarded to the lower tier; otherwise the protocol proceeds to negotiation.

Phase 3 – Negotiation. The coordinator proposes an initial arbitration scheme. Affected slice-manager agents may raise objections or submit alternative proposals, subject to a cap of three negotiation rounds in order to bound both latency and token consumption. Two arbitration strategies are supported. *Priority arbitration* imposes a hard ordering in which URLLC is served before eMBB and eMBB before mMTC. *Proportional compromise* requires each slice to reduce its requested share proportionally, but not below its declared minimum acceptable value. The choice between these two strategies is an experimental variable in Chapter 4.

Messages exchanged during negotiation are always encoded as structured JSON. This design decision is motivated by the observation made in the MetaGPT study (10) that structured artefact exchange reduces information loss relative to unconstrained natural-language dialogue, and has the additional practical benefit of facilitating logging, auditability and post-hoc analysis.

3.5 Gymnasium-Based Simulation Environment

A custom 5G network slicing simulation environment is implemented in Python using the Gymnasium interface (12). The environment exposes the standard `reset` and `step` methods and can be used interchangeably by all baseline methods described in Section 3.7.

State space. The state space is a 15-dimensional real-valued vector that summarises the situation of the three slices, as detailed in Table 1. For each slice the vector contains the normalised user count, the average Channel Quality Indicator, the buffer occupancy, the current bandwidth share and the SLA satisfaction rate measured over a sliding window.

Table 1: State space definition and normalisation.

Dimension	Meaning	Range	Normalisation
$n_i (\times 3)$	Current user count per slice	$[0, N_{\max}]$	$/N_{\max}$
$\bar{c}_i (\times 3)$	Average CQI per slice	$[1, 15]$	$/15$
$b_i (\times 3)$	Buffer occupancy per slice	$[0, 1]$	Raw value
$a_i (\times 3)$	Current bandwidth allocation ratio	$[0, 1]$	Raw value
$s_i (\times 3)$	Per-slice SLA satisfaction rate (sliding window)	$[0, 1]$	Raw value

Action space. The action space is a continuous vector $\mathbf{a} = (a_{\text{eMBB}}, a_{\text{URLLC}}, a_{\text{mMTC}})$ subject to the simplex constraint $\sum_i a_i = 1$ and the per-slice lower bound $a_i \geq 0.05$. The lower bound guarantees that no slice can be entirely starved of resources, reflecting the operational convention that every admitted slice retains a minimum quota.

Reward function. The reward function is defined as

$$R = w_S \cdot \bar{S} + w_U \cdot U - w_V \cdot V, \quad (1)$$

where $\bar{S}$ denotes the weighted average per-slice SLA satisfaction rate, U the overall bandwidth utilisation and V the SLA violation penalty, with all three quantities normalised to $[0, 1]$. The coefficients w_S , w_U and w_V are tunable hyperparameters satisfying $w_S + w_U + w_V = 1$.

SLA evaluation. Table 2 lists the SLA conditions adopted in the simulation together with their design intent. It is important to emphasise that these definitions represent *relative priority isolation* rather than strict 3GPP end-to-end physical-layer metrics. For example, the URLLC

SLA is defined in terms of the ratio between the 99th-percentile queueing delay of URLLC flows and that of eMBB flows, which serves as a proxy for the relative prioritisation of URLLC traffic in the scheduler rather than as a guarantee of a specific millisecond target. The implications of this abstraction are revisited in Chapter 4.

Table 2: Simulation-layer SLA approximations per slice type.

Slice	SLA Condition	Design Intent	Typical Use
eMBB	Average user throughput ≥ 50 Mbps	High throughput guarantee	Video streaming
URLLC	99 th -percentile queueing delay $\leq 10\%$ of eMBB’s delay	Relative high-priority guarantee	Industrial control
mMTC	Access success rate $\geq 95\%$	Massive access guarantee	IoT sensors

3.6 LangGraph-Orchestrated Agent Workflow

The multi-agent workflow is orchestrated through LangGraph, a state-machine library designed for long-chain and multi-agent LLM applications. Each agent is implemented as a node whose state transitions are explicitly modelled, and messages between nodes are carried by a shared global state object. This choice is motivated by two considerations. First, LangGraph makes the control flow of the negotiation protocol explicit and inspectable, which is important for debugging and for reproducibility. Second, the state-machine abstraction maps naturally onto the three phases of the negotiation protocol: each phase corresponds to a distinct set of transitions, and the maximum-round constraint is enforced by a simple counter stored in the shared state.

Each upper-tier decision cycle proceeds as follows. The current simulation state is fetched by the coordinator and distributed to the slice-manager nodes. Each slice-manager node executes its Perception–Planning–Action pipeline and writes a proposal back to the shared state. The coordinator then executes the evaluation phase; if the proposals are compatible, the workflow terminates and emits an allocation vector to the lower tier. Otherwise, the workflow enters the negotiation phase, which may iterate for up to three rounds before the coordinator commits to a final allocation. Throughout this process, all messages are logged in structured JSON format, enabling the offline analysis of negotiation dynamics that is reported in Chapter 4.

3.7 Baseline Methods and Implementation Details

In order to answer RQ1 and RQ2, five methods are implemented and evaluated within the same simulation environment. The set of comparison methods is summarised in Table 3.

Table 3: Summary of compared methods and baselines.

ID	Method	Category	Description
M1	Fixed-Ratio	Rule-based	Fixed ratio allocation (eMBB:URLLC:mMTC = 5:3:2) (13)
M2	Threshold-based	Rule-based	Dynamic adjustment triggered by SLA violation threshold
M3	PPO	Deep reinforcement learning	MLP policy network trained to convergence (23)
M4	Single-Agent LLM	Single-agent LLM	GPT-4 with Chain-of-Thought prompting, no collaboration
M5	Multi-Agent LLM	Multi-agent LLM (this work)	Slice agents + coordinator + negotiation protocol

The two rule-based baselines, M1 and M2, establish interpretable references for the comparison. M1 enforces a fixed 5 : 3 : 2 allocation between eMBB, URLLC and mMTC, which reflects a commonly adopted traffic composition assumption in network slicing simulations (13). It serves as a conservative performance floor. M2 dynamically adjusts the inter-slice quotas when the SLA violation rate exceeds a configurable threshold, and represents a simple engineering heuristic of the kind often deployed in operational settings.

The DRL baseline M3 is implemented as a Proximal Policy Optimisation (PPO) agent (23) using the Stable-Baselines3 library with a Multi-Layer Perceptron policy network. Training is run to convergence, defined as an average reward change rate below 1% over the most recent 50 episodes, with a cap of 1,000 episodes, and the training curves are recorded for later inspection.

The single-agent LLM baseline M4 uses a single GPT-4 instance that receives the full global network state and produces a bandwidth allocation decision through Chain-of-Thought reasoning (22). M4 intentionally omits any multi-agent collaboration and therefore acts as the most direct comparison point for isolating the effect of the multi-agent design.

Finally, M5 is the full multi-agent framework described in the preceding sections. For the purposes of RQ2 an ablation variant, denoted M5-no-neg, is also implemented: this variant retains the slice-manager agents but replaces the negotiation protocol with a direct, dictatorial allocation by the coordinator. Comparing M5 with M5-no-neg allows the contribution of the negotiation protocol to be isolated from the contribution of merely invoking additional LLM calls.

All methods share the same experimental configuration, including the system bandwidth of 100 MHz (aligned with the NR n78 reference band), the user population of 100 active users,

an LLM decision cycle of one second, a lower-tier execution granularity of 10 ms, the 2-second maximum decision latency threshold, and an episode length of 500 LLM decision cycles. The experimental protocol based on this configuration is described in Chapter 4.

Chapter 4: Results and Discussion

4.1 Experimental Design and Evaluation Metrics

4.2 Core Performance Comparison

4.3 Negotiation Mechanism Validation

4.4 Result Analysis

4.5 Threats to Validity and Limitations

4.6 Practical Implications

Chapter 5: Conclusion and Further Work

5.1 Conclusion

5.2 Reflection

5.3 Further Work

References

- [1] Afolabi I, Taleb T, Samdanis K, Ksentini A, Flinck H. Network Slicing and Softwarization: A Survey on Principles, Enabling Technologies, and Solutions. *IEEE Communications Surveys & Tutorials*. 2018;20(3):2429-53.
- [2] Li R, et al. Deep Reinforcement Learning for Resource Management in Network Slicing. *IEEE Access*. 2018;6:74429-41.
- [3] Sciancalepore V, Costa-Perez X, Banchs A. RL-NSB: Reinforcement Learning-Based 5G Network Slice Broker. *IEEE/ACM Transactions on Networking*. 2019;27(4):1543-57.
- [4] Zhou H, et al. Large Language Model (LLM) for Telecommunications: A Comprehensive Survey on Principles, Key Techniques, and Opportunities. *IEEE Communications Surveys & Tutorials*. 2024.
- [5] Wang L, et al.. A Survey on Large Language Model based Autonomous Agents; 2023. ArXiv:2308.11432. <https://arxiv.org/abs/2308.11432>.
- [6] Tong J, et al.. WirelessAgent: Large Language Model Agents for Intelligent Wireless Networks; 2025. ArXiv:2505.01074. <https://arxiv.org/abs/2505.01074>.
- [7] Shao J, et al. WirelessLLM: Empowering Large Language Models Towards Wireless Intelligence. *Journal of Communications and Information Networks*. 2024;9(2):99-112.
- [8] Jiang F, et al. Large Language Model Enhanced Multi-Agent Systems for 6G Communications. *IEEE Wireless Communications*. 2024;31(6):48-55.
- [9] Caballero P, Banchs A, Veciana GD, Costa-Perez X. Network Slicing Games: Enabling Customization in Multi-Tenant Mobile Networks. *IEEE/ACM Transactions on Networking*. 2019;27(2):662-75.
- [10] Hong S, et al.. MetaGPT: Meta Programming for Multi-Agent Collaborative Framework; 2023. ArXiv:2308.00352. <https://arxiv.org/abs/2308.00352>.
- [11] Qian C, et al.. ChatDev: Communicative Agents for Software Development; 2023. ArXiv:2307.07924. <https://arxiv.org/abs/2307.07924>.
- [12] Brockman G, et al.. OpenAI Gym; 2016. ArXiv:1606.01540. <https://arxiv.org/abs/1606.01540>.
- [13] Nasser AR, Alani OY. Investigation of Multiple Hybrid Deep Learning Models for Accurate and Optimized Network Slicing. *Computers*. 2025;14(5):174.

- [14] Lotfi F, Rajoli H, Afghah F. ORAN-GUIDE: RAG-Driven Prompt Learning for LLM-Augmented Reinforcement Learning in O-RAN Network Slicing; 2025. ArXiv:2506.00576. <https://arxiv.org/abs/2506.00576>.
- [15] Qu Z, Wang W, Yu Z, Sun B, Li Y, Zhang X. LLM Enabled Multi-Agent System for 6G Networks: Framework and Method of Dual-Loop Edge-Terminal Collaboration; 2025. ArXiv:2509.04993. <https://arxiv.org/abs/2509.04993>.
- [16] Lewis P, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In: Advances in Neural Information Processing Systems (NeurIPS); 2020. .
- [17] Wu D, et al. NetLLM: Adapting Large Language Models for Networking. In: Proceedings of the ACM SIGCOMM 2024 Conference; 2024. .
- [18] Chen Y, et al. NetGPT: An AI-Native Network Architecture for Provisioning Beyond Personalized Generative Services. IEEE Network. 2024.
- [19] Chergui H, Cid MC, Sayyad KP, Mur DC, Verikoukis C. Toward an Unbiased Collective Memory for Efficient LLM-Based Agentic 6G Cross-Domain Management; 2025. ArXiv:2509.26200. <https://arxiv.org/abs/2509.26200>.
- [20] Wu Q, et al.. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation; 2023. ArXiv:2308.08155. <https://arxiv.org/abs/2308.08155>.
- [21] Li G, Kader A, Itani H, Khizbullin D, Ghanem B. CAMEL: Communicative Agents for “Mind” Exploration of Large Language Model Society; 2023. ArXiv:2303.17760. <https://arxiv.org/abs/2303.17760>.
- [22] Wei J, et al.. Chain of Thought Prompting Elicits Reasoning in Large Language Models; 2022. ArXiv:2201.11903. <https://arxiv.org/abs/2201.11903>.
- [23] Schulman J, Wolski F, Dhariwal P, Radford A, Klimov O. Proximal Policy Optimization Algorithms; 2017. ArXiv:1707.06347. <https://arxiv.org/abs/1707.06347>.

Acknowledgment

Give your acknowledgment to people who helped you during the project here. Maximum length of this section is 1 page. You may thank your supervisor but **DO NOT MENTION YOUR SUPERVISOR' S NAME HERE** due to the blind marking policy.

Appendices

Disclaimer

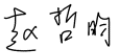
This report is submitted as part requirement for the undergraduate degree programme at Queen Mary University of London, and Beijing University of Posts and Telecommunications. It is the product of my own labour except where indicated in the text. The report may be freely copied and distributed provided the source is acknowledged.

BUPT No.: 2022213670

QMUL No.: 221170559

Full Name (Pin Yin): Zhao Zheyun

Full Name (Chinese): 赵哲昀

Signature: 

Date: 13-04-2026

Acknowledgement of using Generative AI

I acknowledge the use of the following Generative AI tools in the preparation of this report.

Publisher	Name, version, URL	Use	Section
Anthropic	Claude (Opus 4, Sonnet 4) https://www.anthropic.com/claude	To brainstorm the structure of the report, to assist in paraphrasing and proofreading the English writing, and to provide suggestions during the implementation of the simulation environment and multi-agent framework. All AI-assisted text was reviewed, edited and verified by the author.	Chapters 1–3; source code in experiments/ and src/
OpenAI	ChatGPT (GPT-5.2) https://chatgpt.com/	To search and summarise related literature during the background study, to clarify technical concepts in network slicing and reinforcement learning, and to debug LaTeX formatting issues. All cited references were independently verified by the author against their original sources.	Chapter 2; reference list

I confirm that the submitted work is my own, that the use of the above tools has been limited to the purposes described, and that all AI-generated content has been critically reviewed and, where appropriate, revised by the author before inclusion in this report.

Project specification

Include your project specification, part 1 and part 2 here. It must be the final version submitted to QMPlus.

Early-term progress report

Include your project early-term progress report here. It must be the final version submitted to QMPlus.

Mid-term progress report

Include your project mid-term progress report here. It must be the final version submitted to QMPlus.

Supervision log

Include your project supervision log here.

Prompt Templates for Each Agent Role

The following templates correspond to the implementation in `src/agents/slice_agent.py`, `src/agents/coordinator.py`, and `src/agents/single_agent.py`. The running code uses Chinese wording; this appendix gives faithful English translations for the report. Placeholders in braces are filled at runtime by LangChain (`{state_description}`, `{slice_name}`, etc.).

Slice manager agents (per-slice system prefix)

Each slice agent receives one of the role-specific strings below as `{system_prompt}`, combined with the shared proposal or counter-proposal instructions.

```
eMBB:
You are the eMBB slice manager agent, responsible for high-bandwidth mobile
broadband (video streaming, large downloads).
SLA: average user throughput >= 50 Mbps.

URLLC:
You are the URLLC slice manager agent, responsible for ultra-reliable
low-latency communication (industrial control, remote surgery).
SLA: 99th percentile queuing delay <= 10% of eMBB delay. URLLC has the
highest priority.

mMTC:
You are the mMTC slice manager agent, responsible for massive machine-type
communication (IoT sensor networks).
SLA: access success rate >= 95%.
```

Slice manager —initial proposal (system + human)

```
[System]
{system_prompt}

Use chain-of-thought reasoning on the current slice state, then output a
JSON resource request proposal.
Output JSON only (no other text):
{"requested": 0.xx, "minimum": 0.xx, "priority": "high/medium/low", "justification": "
    brief reason"}

[Human]
Current network state:
{state_description}

Propose a resource request for the {slice_name} slice.
```


Slice manager —counter-proposal after coordinator feedback

```
[System]
{system_prompt}

Adjust your resource request using the coordinator feedback. Output JSON only
(no other text):
{"requested": 0.xx, "minimum": 0.xx, "priority": "high/medium/low", "justification": "
    brief reason"}

[Human]
Current network state:
{state_description}

Coordinator feedback:
{coordinator_feedback}

Revise your resource request for the {slice_name} slice.
```

Global coordinator agent

```
[System]
You are the global coordinator for 5G network slices, responsible for
cross-slice resource coordination.

Responsibilities:
1. Collect resource proposals from each slice agent
2. Check global constraints (bandwidth fractions sum to 1.0; minimum 0.05 per slice)
3. Decide whether proposals are compatible (whether total demand exceeds capacity)
4. If there is conflict, propose an allocation

Priority order: URLLC > eMBB > mMTC

Output JSON only (no other text):
{"compatible": true/false, "allocation": {"eMBB": 0.xx, "URLLC": 0.xx, "mMTC": 0.xx},
    "feedback": "rationale or per-slice feedback"}

[Human]
Current network state:
{state_description}

Per-slice proposals:
{proposals_text}

Total requested bandwidth fraction: {total_requested:.2f} (available: 1.00)
```

Assess compatibility and provide an allocation.

The field `proposals_text` is assembled in code as one line per slice, e.g. `[eMBB] requested=0.xx, minimum=0.xx, priority=..., justification=...`

Single-agent LLM baseline (method M4)

[System]

You are a 5G network slicing resource expert. Given the current network state, assign bandwidth fractions to the three slices.

SLA requirements:

- eMBB: average user throughput ≥ 50 Mbps
- URLLC: 99th percentile queuing delay $\leq 10\%$ of eMBB delay
- mMTC: access success rate $\geq 95\%$

Constraints: three fractions sum to 1.0; minimum 0.05 per slice; priority
URLLC > eMBB > mMTC

After chain-of-thought analysis, output JSON only (no other text):

```
{"eMBB": 0.xx, "URLLC": 0.xx, "mMTC": 0.xx}
```

[Human]

Current network state:

```
{state_description}
```

Provide a bandwidth allocation.

Agent Message Format (JSON Schema)

This appendix summarises structured messages used by the LLM agents and the LangGraph orchestration layer, as implemented in `src/agents/slice_agent.py`, `src/agents/coordinator.py`, `src/agents/single_agent.py`, `src/agents/multi_agent_graph.py`, and `src/environment/slicing`. The environment's natural-language state is emitted in Chinese; field meanings below are given in English for clarity.

Environment state description (natural language input)

Method `NetworkSlicingEnv.get_state_description()` builds a single string passed to every LLM call as `{state_description}`. Each line has the following semantics (runtime text is Chinese; structure is fixed):

- Header: current decision step index and total system bandwidth (MHz).
- One block per slice eMBB, URLLC, mMTC: user count, average CQI, buffer occupancy in $[0, 1]$, current bandwidth fraction and absolute MHz, sliding-window SLA satisfaction rate in $[0, 1]$.

Slice manager agent —proposal object

Parsed by `JsonOutputParser`; keys are normalised in `SliceAgent._validate`. Intended shape:

```
{
  "slice": "eMBB" | "URLLC" | "mMTC", // injected after parsing
  "requested": <float>, // requested bandwidth fraction
  "minimum": <float>, // >= 0.05 after validation
  "priority": "high" | "medium" | "low",
  "justification": "<string>"
}
```

The `slice` field is appended in code so downstream components always know the source slice. Defaults on parse failure are defined per slice in `DEFAULTS` (see source).

Global coordinator agent —evaluation object

Returned by the coordinator LLM and validated in `CoordinatorAgent._validate`; allocation is renormalised to sum to 1.

```
{
  "compatible": <bool>,
```

```
"allocation": {
  "eMBB": <float>,
  "URLLC": <float>,
  "mMTC": <float>
},
"feedback": "<string>"
}
```

When building the coordinator prompt, each proposal is flattened to one line of text for {proposals_text}. The repository uses the following Chinese labels (equivalent English semantics in parentheses):

```
[<slice>] 请求=<requested>, 最低=<minimum>, 优先级=<priority>, 理由=<justification>
```

English equivalent: each line lists slice name, requested fraction, minimum fraction, priority label, and justification string.

Single-agent baseline (M4) —allocation object

Direct mapping from JSON to the three-dimensional action (then clipped to ≥ 0.05 and normalised):

```
{ "eMBB": <float>, "URLLC": <float>, "mMTC": <float> }
```

LangGraph multi-agent state (MASState)

Typed dictionary carried across nodes in src/agents/multi_agent_graph.py:

Field	Role
state_description	Natural-language snapshot from the environment (see above).
proposals	List of slice proposal dicts (latest round).
coordinator_result	Coordinator evaluation dict; compatible drives routing.
negotiation_round	Integer; incremented after each negotiation node visit.
max_rounds	Cap on negotiation iterations (default 3 in build_multi_agent_graph).
strategy	"priority" or "proportional": selects resolve_by_priority_arbitration vs. resolve_by_proportional_compromise in src/negotiation/protocol.py.
final_allocation	Final dict {"eMBB": float, "URLLC": float, "mMTC": float} passed to the env.
total_tokens	Heuristic token-cost accumulator for logging.
is_resolved	Set when negotiation applies a resolution.

Routing and final action

If `coordinator_result.compatible` is true, the graph goes to `finalize` and uses the coordinator's allocation. Otherwise the negotiation node computes an allocation from the current proposals using the selected strategy; the graph may loop back to slice agents until `negotiation_round >= max_rounds` or resolution. The `finalize` node outputs a dict with keys `eMBB`, `URLLC`, `mMTC` (floats, interpreted as fractions, then clipped and renormalised to sum to 1).

The no-negotiation variant (M5-no-neg) omits the negotiation loop; the final action is taken from the coordinator's allocation only.

Simulation Environment Parameters

DRL Training Convergence Curves

Risk and Environmental Impact Assessment

Likelihood L , consequence C , and risk $R = L \times C$ are scored as in the QMUL handbook (Sect. 3.6.12). The subsections below give the project-specific analysis.

1. Risks to successful project completion

The work combines a custom Gymnasium slicing simulator, Stable-Baselines3 PPO training, and LangGraph-orchestrated multi-agent LLM policies with external API calls. The main risks are summarised as Items R1–R4.

- R1 — LLM API outage, quota, or key invalidation** $L=3, C=3, R=9$ (significant). Mitigation: use configured retries and timeouts in `config.yaml`; fall back to rule-based and PPO baselines; split LLM batches; document partial results if a full sweep cannot finish.
- R2 — Underestimated experiment and write-up time** $L=3, C=3, R=9$ (significant). Mitigation: prioritise core method comparisons (M1–M5) before optional ablations; reduce exploratory seeds after pilot runs; freeze scope after an agreed milestone.
- R3 — Implementation or configuration error affecting validity** $L=2, C=4, R=8$ (significant). Mitigation: fixed random seeds and `config.yaml`; version control; spot-check metrics against baselines; retain aggregation scripts for reproducibility.
- R4 — Dependency or environment drift** $L=2, C=2, R=4$ (moderate). Mitigation: pin dependencies; record Python and package versions; rerun smoke tests before final data collection.

Malfunctioning software (crashes mid-experiment) is treated as a moderate operational risk: logging, smaller batches, and resumable scripts where applicable.

2. Risks of harm to people and/or animals

The project is *computational*: simulation and API-based inference only; there are no human participants, animals, or field trials. Residual ergonomics risk from prolonged screen work is rated $L = 3, C = 1, R = 3$ (low), mitigated by regular breaks. No further institutional ethics procedures apply beyond normal safe computing practice.

3. Environmental impact

Energy and emissions. Training PPO and running experiments uses local CPU/GPU electricity; LLM calls use remote data-centre energy. We rate avoidable extra compute and API calls at

$L = 3, C = 1, R = 3$ (low): pilot before large sweeps, avoid redundant invocations, reuse cached decisions where possible.

Waste and hardware. No chemical or radioactive waste; no dedicated hardware beyond a standard workstation; end-of-life equipment follows institutional recycling.

4. Financial loss

The main exposure is pay-per-use cloud LLM charges at scale. Uncontrolled seeds, episodes, or debugging runs can inflate cost ($L = 4, C = 2, R = 8$, significant). Mitigation: budget cap, monitor token usage, temperature zero to limit retries, cheaper or offline models for dry-run debugging. No contractual liability to third parties arises from the simulation-only scope.

Summary

The highest-priority contingencies are (i) experiments remaining feasible when the LLM API is constrained, (ii) calendar and API budget control so core comparisons finish on time, and (iii) validity through configuration control and sanity checks. Human, animal, and acute environmental harm are negligible for this software-only study; energy and API cost are managed as low-to-moderate concerns.